INESCTEC - UAV Real-Time Tracking

Tomás Barros

May 2026 to July 2026

A Real-Time Autonomous UAV Repositioning Framework for Maintaining Line-of-Sight Communication in Obstacle-Rich Environments

Tomás Barros

Abstract

Reliable wireless communication is one of the primary challenges faced by autonomous unmanned aerial vehicles (UAVs) operating in urban and cluttered environments. Physical obstacles frequently obstruct the direct Line-of-Sight (LoS) between the aerial platform and ground users, resulting in communication degradation and reduced Quality of Service (QoS). This paper presents a real-time autonomous UAV repositioning framework capable of continuously adapting the vehicle's position to preserve LoS while avoiding environmental obstacles.

The proposed system combines a pre-mapped occupancy grid representation with real-time user localization to determine the optimal UAV position using the Traffic- and Obstacle-aware Positioning Algorithm (TOPA). Once a new target position has been identified, a two-stage motion planning strategy is employed. First, the Lazy Theta* algorithm computes a collision-free any-angle path through the environment. Subsequently, cubic B-Spline interpolation generates a smooth trajectory compatible with the UAV's kinematic constraints. The entire framework has been implemented within a ROS2-based software architecture and integrated with MAVLink communication for autonomous flight execution in an ArduPilot Software-in-the-Loop (SITL) simulation environment.

Experimental results demonstrate that the proposed framework successfully maintains communication visibility while producing dynamically feasible trajectories suitable for real-time autonomous navigation. The modular architecture also enables future deployment on physical UAV platforms with minimal modifications.

Autonomous UAV, Path Planning, ROS2, MAVLink, Line-of-Sight, Occupancy Grid, Lazy Theta*, B-Spline, TOPA, Quality of Service

1 Introduction

This documentation is a simplified version of the actual work, it was made by IA to have at least something written and not only simulated. Autonomous

unmanned aerial vehicles (UAVs) have become an increasingly important component of modern robotic systems. Their capability to rapidly deploy sensing, communication and monitoring services makes them suitable for applications including emergency response, industrial inspection, surveillance, environmental monitoring and temporary communication infrastructures. As UAV autonomy increases, maintaining reliable communication between the aerial platform and mobile ground users becomes a critical requirement.

Most wireless communication technologies employed by UAVs rely heavily on maintaining a direct Line-of-Sight (LoS) between transmitter and receiver. In dense urban environments or confined industrial facilities, buildings, walls and other obstacles frequently obstruct the communication path, leading to packet losses, increased latency and significant degradation of the Quality of Service (QoS). Consequently, preserving LoS is not simply a communication problem but also a motion planning problem, requiring the UAV to continuously reposition itself according to the user's movement and the surrounding environment.

Traditional autonomous navigation algorithms primarily focus on obstacle avoidance and shortest-path optimisation without explicitly considering communication quality during trajectory generation. While these approaches ensure safe navigation, they often produce trajectories that temporarily lose visibility of the communication target. Conversely, communication-aware positioning methods frequently neglect the dynamic constraints required for safe autonomous flight.

This work addresses both challenges simultaneously by integrating communication-aware positioning with autonomous trajectory planning into a unified framework. The proposed system continuously estimates the optimal UAV location capable of preserving LoS with a moving ground user while generating collision-free trajectories that satisfy the vehicle's motion constraints.

The proposed framework consists of four main components:

- A pre-mapped occupancy grid describing the navigable environment.
- A real-time positioning module based on the Traffic- and Obstacle-aware Positioning Algorithm (TOPA), responsible for selecting the optimal UAV destination.
- A global path planner based on the Lazy Theta* algorithm, capable of computing short any-angle collision-free paths.

- A trajectory smoothing stage using cubic B-Spline interpolation, producing dynamically feasible trajectories suitable for autonomous execution.

The complete framework has been implemented using ROS2 and integrated with the ArduPilot Software-in-the-Loop (SITL) simulator through the MAVLink communication protocol. This architecture enables real-time testing of autonomous repositioning strategies while remaining fully compatible with future deployment on physical UAV platforms.

The main contributions of this work are summarised as follows:

1. The design of a modular communication-aware autonomous navigation architecture for UAVs.
2. The integration of TOPA with real-time autonomous motion planning.
3. The combination of Lazy Theta* path planning with B-Spline trajectory generation for smooth navigation.
4. A complete ROS2 implementation integrated with MAVLink and ArduPilot SITL.
5. Experimental validation demonstrating the feasibility of maintaining Line-of-Sight communication in obstacle-rich environments.

The remainder of this paper is organised as follows. Section II presents the overall system architecture. Section III describes the communication-aware positioning strategy based on TOPA. Section IV introduces the motion planning pipeline, including occupancy grid generation, Lazy Theta* path planning and trajectory smoothing. Section V discusses the software implementation and system integration. Experimental validation and performance analysis are presented in Sections VI and VII, followed by the conclusions and future research directions.

2 System Architecture

The proposed framework has been designed as a modular robotic architecture capable of autonomously repositioning an unmanned aerial vehicle (UAV) to preserve wireless communication with a mobile ground user while

safely navigating through obstacle-rich environments. The overall system combines communication-aware positioning, global path planning and autonomous flight control into a unified software pipeline implemented in ROS2.

Unlike conventional navigation frameworks, which treat communication and motion planning as independent problems, the proposed architecture tightly couples both components. Every time the ground user's position changes, the communication module evaluates the current visibility conditions and computes a new optimal UAV location. The navigation subsystem subsequently generates a collision-free trajectory towards this destination while respecting the UAV's kinematic constraints.

Figure **??** illustrates the overall architecture of the proposed framework. The system is organised into five independent modules:

1. Environment Representation
2. Communication-Aware Positioning
3. Global Path Planning
4. Trajectory Generation
5. Flight Control and Execution

Each module performs a clearly defined task while communicating through ROS2 topics and services. This modular design simplifies future upgrades, allowing individual components to be replaced without affecting the remaining architecture.

2.1 Environment Representation

The environment is represented using a two-dimensional occupancy grid generated from a previously mapped indoor scenario. Each cell of the grid stores its occupancy state, distinguishing between free space and obstacles.

This representation offers several advantages. First, occupancy grids provide a computationally efficient structure for collision detection during path planning. Second, they allow rapid visibility analysis between arbitrary points in the environment, which is essential for evaluating Line-of-Sight (LoS) conditions. Finally, occupancy grids are fully compatible with standard robotic planning algorithms available in ROS2.

Although the current implementation assumes a static environment, the proposed architecture has been designed such that dynamic obstacle updates can be incorporated by continuously modifying the occupancy probabilities of affected cells.

2.2 Communication-Aware Positioning

Maintaining reliable communication requires more than simply following the shortest path to the user. Instead, the UAV must continuously search for positions that maximise communication quality while remaining physically reachable.

The positioning module receives the estimated ground-user location together with the occupancy grid describing the environment. Based on these inputs, the Traffic- and Obstacle-aware Positioning Algorithm (TOPA) evaluates multiple candidate positions around the user.

Each candidate position is analysed according to several criteria:

- Existence of unobstructed Line-of-Sight.
- Distance to the ground user.
- Estimated communication quality.
- Obstacle proximity.
- Navigation feasibility.

The highest-scoring candidate becomes the desired UAV destination. Since this optimisation is repeated whenever the user moves, the framework naturally adapts to changing operating conditions without requiring predefined trajectories.

2.3 Global Path Planning

Once the optimal destination has been determined, the navigation subsystem computes a collision-free path connecting the UAV's current position to the target location.

Instead of relying on traditional grid-constrained search algorithms, the proposed framework employs Lazy Theta*, an any-angle extension of the classical A* algorithm.

Unlike conventional A*, which restricts movement to neighbouring grid cells, Lazy Theta* allows direct connections whenever two nodes share unobstructed visibility. Consequently, the generated paths are considerably shorter and more natural, reducing unnecessary heading changes and flight time.

The planner performs graph expansion directly over the occupancy grid while continuously validating Line-of-Sight between candidate nodes. This strategy significantly reduces the number of turns compared to standard grid-based planning methods.

2.4 Trajectory Generation

Although Lazy Theta* produces geometrically efficient paths, the resulting waypoint sequence still contains discontinuities in heading and curvature that cannot be directly followed by a real UAV.

To generate dynamically feasible trajectories, the waypoint sequence undergoes a second processing stage based on cubic B-Spline interpolation.

The smoothing process preserves the overall geometry of the planned path while introducing continuous first- and second-order derivatives. Consequently, the generated trajectories exhibit significantly smoother velocity and acceleration profiles, reducing aggressive manoeuvres and improving overall flight stability.

The resulting trajectory consists of densely sampled reference points that can be directly transmitted to the flight controller.

2.5 Flight Control and Software Integration

The entire framework has been implemented using the Robot Operating System 2 (ROS2), where each functional block executes as an independent node.

The communication between ROS2 and the autopilot is performed through the MAVLink protocol, allowing high-level trajectory commands to be translated into low-level flight instructions.

The proposed implementation includes dedicated nodes responsible for:

- Receiving telemetry from the UAV.
- Monitoring the user's position.
- Executing the TOPA optimisation.

- Computing global paths.
- Generating smooth trajectories.
- Sending waypoint commands to ArduPilot.
- Logging system performance for offline analysis.

This distributed architecture enables each subsystem to execute asynchronously while maintaining deterministic communication through ROS2 middleware.

2.6 Operational Pipeline

The complete execution pipeline follows the sequence illustrated below:

1. The user's current position is received.
2. The occupancy grid is queried.
3. TOPA computes the optimal communication position.
4. Lazy Theta* generates a collision-free path.
5. B-Spline interpolation smooths the trajectory.
6. The trajectory is transmitted through MAVLink.
7. The UAV autonomously executes the manoeuvre.
8. The process repeats whenever the user changes position.

This closed-loop strategy allows the UAV to continuously adapt its position throughout the mission, maintaining reliable communication while safely avoiding environmental obstacles.

One of the main strengths of the proposed architecture lies in the clear separation between communication optimisation and motion planning. Consequently, future work may replace individual algorithms—such as TOPA or Lazy Theta*—without requiring substantial modifications to the remaining software components.

3 Communication-Aware UAV Positioning

Maintaining an unobstructed communication link between an unmanned aerial vehicle (UAV) and a mobile ground user requires continuously adapting the UAV position as the surrounding environment changes. Unlike conventional navigation systems, where the destination is predefined, the destination in the proposed framework is itself the output of an optimisation process.

The objective of the positioning module is therefore to determine, in real time, the UAV location that maximises communication quality while remaining safely reachable and compatible with the navigation subsystem.

3.1 Problem Definition

Consider a bounded environment represented by an occupancy grid

$$\mathcal{M} = \{m_{ij}\},$$

where each cell is classified as either free or occupied.
Let

$$\mathbf{u} = (x_u, y_u)$$

represent the current position of the ground user and

$$\mathbf{p} = (x_p, y_p)$$

the current UAV position.
The goal is to determine a new UAV destination

$$\mathbf{p}^*$$

such that communication quality is maximised while avoiding collisions with environmental obstacles.

Unlike shortest-path problems, the optimal destination is not necessarily the closest point to the user. Instead, several candidate positions are evaluated according to both communication and navigation criteria.

3.2 Candidate Position Generation

The first stage of the algorithm generates a discrete set of candidate positions surrounding the ground user.

Given a search radius R, candidate locations are uniformly distributed around the user

$$\mathcal{C} = \{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_N\}.$$

Each candidate represents a potential UAV destination from which the communication link could be established.

Positions located inside occupied cells are immediately discarded since they cannot be reached safely.

The remaining candidates are then evaluated independently.

3.3 Line-of-Sight Verification

Reliable wireless communication strongly depends on maintaining direct Line-of-Sight (LoS) between transmitter and receiver.

For every candidate position, a visibility test is performed using the occupancy grid.

Let

$$LOS(\mathbf{u}, \mathbf{c}_i)$$

be a binary visibility function defined as

$$LOS(\mathbf{u}, \mathbf{c}_i) = \begin{cases} 1, & \text{if no obstacle intersects the line segment} \\ 0, & \text{otherwise.} \end{cases}$$

Visibility testing is performed by discretising the line connecting both points and verifying whether any occupied grid cell intersects the segment.

Candidates that fail this test receive the lowest possible score and are removed from further evaluation.

3.4 Candidate Evaluation

For each visible candidate, the algorithm evaluates several metrics that jointly characterise its suitability.

The current implementation considers four independent criteria:

- communication visibility;
- distance to the user;
- distance from nearby obstacles;
- navigation feasibility.

These quantities are combined into a single objective function

$$J(\mathbf{c}_i) = w_1 V_i + w_2 D_i + w_3 S_i + w_4 F_i,$$

where

- V_i denotes the visibility score;
- D_i measures the communication distance;
- S_i represents obstacle clearance;
- F_i estimates navigation feasibility;
- w_k are weighting coefficients.

The weighting factors can be tuned according to the operational requirements of the mission.

For example, surveillance applications may prioritise communication reliability, whereas emergency inspection tasks may favour shorter repositioning times.

3.5 Optimal Position Selection

After evaluating every candidate, the optimal destination is selected as

$$\mathbf{p}^* = \arg\max_{\mathbf{c}_i \in \mathcal{C}} J(\mathbf{c}_i).$$

This optimisation is computationally inexpensive because only a finite number of candidate locations are analysed.

Consequently, the algorithm can be executed repeatedly as the ground user moves, enabling continuous online repositioning throughout the mission.

3.6 TOPA Algorithm

Algorithm 3.6 summarises the complete optimisation procedure.

[H] Traffic- and Obstacle-aware Positioning Algorithm (TOPA)
Occupancy Grid $\mathcal{M}$, User Position $\mathbf{u}$
Optimal UAV Position $\mathbf{p}^*$
Generate candidate positions around the user
Discard candidates inside occupied cells
candidate position
Perform Line-of-Sight verification
LoS exists
Evaluate communication score
Evaluate obstacle clearance
Evaluate navigation feasibility
Compute objective function J
Assign minimum score
Return candidate with highest score

3.7 Computational Complexity

Let N denote the number of candidate positions generated around the user.

Each candidate requires a visibility test and the computation of a fixed number of evaluation metrics.

Assuming the occupancy grid lookup is performed in constant time, the overall computational complexity becomes

$$\mathcal{O}(N),$$

making the algorithm suitable for real-time operation.

Since the number of candidates remains relatively small in practical deployments, the optimisation stage introduces negligible computational overhead when compared to the subsequent path-planning process.

3.8 Discussion

The main advantage of TOPA lies in separating communication optimisation from trajectory generation.

Instead of directly planning towards the user's current position, the algorithm first determines the most suitable communication location and only then invokes the navigation subsystem.

This hierarchical strategy significantly reduces unnecessary path replanning and naturally incorporates communication constraints into the autonomous navigation pipeline.

Moreover, because the optimisation relies exclusively on occupancy-grid information, the proposed approach remains independent of the underlying path planner. Consequently, alternative planning algorithms such as A*, D* Lite or RRT* could replace Lazy Theta* without requiring modifications to the positioning stage.

4 Motion Planning

After determining the optimal communication position, the UAV must autonomously navigate towards the selected destination while avoiding collisions with the surrounding environment. Since the destination computed by the positioning module may be separated from the current UAV location by several obstacles, a dedicated motion-planning pipeline is required.

The proposed framework decomposes the navigation problem into two consecutive stages. First, a global planner computes a collision-free path over the occupancy grid using the Lazy Theta* algorithm. Although this planner produces short any-angle paths, the resulting waypoint sequence still contains discontinuities that are unsuitable for direct execution by a quadrotor. Therefore, a second stage performs trajectory smoothing using cubic B-Spline interpolation, producing a continuous trajectory that satisfies the vehicle's kinematic requirements.

Figure **??** illustrates the proposed planning pipeline.

4.1 Occupancy Grid Representation

Efficient path planning requires a geometric representation of the operating environment. In the proposed framework, the environment is modelled as a two-dimensional occupancy grid in which each cell represents a discrete portion of the workspace.

Formally, the occupancy grid is defined as

$$\mathcal{M} : R^2 \rightarrow \{0, 1\},$$

where

$$\mathcal{M}(x, y) = \begin{cases} 0, & \text{free space} \\ 1, & \text{occupied space.} \end{cases}$$

This binary representation enables constant-time collision checking during graph exploration while remaining computationally efficient for large environments.

Each obstacle is expanded using an inflation radius corresponding to the UAV safety margin. This operation ensures that the planner automatically generates trajectories that maintain an adequate clearance from nearby obstacles.

4.2 Graph Construction

The occupancy grid is interpreted as a weighted graph

$$G = (V, E),$$

where every free cell corresponds to a graph vertex and neighbouring cells define graph edges.

Unlike classical graph search methods that restrict movement to horizontal and diagonal neighbours, the proposed framework allows direct connections whenever two vertices share Line-of-Sight visibility.

This significantly reduces unnecessary heading changes and produces paths that better approximate the Euclidean shortest path.

4.3 Lazy Theta* Path Planning

Among existing graph-search algorithms, Lazy Theta* was selected because it combines the robustness of A* with any-angle path generation while reducing the number of expensive visibility tests.

Traditional A* constrains every node expansion to adjacent cells, producing stair-shaped trajectories even in obstacle-free environments. Theta* improves this behaviour by allowing a node to inherit its parent's parent whenever direct visibility exists.

Lazy Theta* further optimises this process by postponing visibility verification until node expansion, thereby reducing computational cost without affecting path quality.

The planner minimises the evaluation function

$$f(n) = g(n) + h(n),$$

where

- $g(n)$ represents the accumulated travel cost;
- $h(n)$ denotes the heuristic estimate to the goal.

The heuristic is computed using the Euclidean distance

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2},$$

which is both admissible and consistent for this problem.

4.4 Line-of-Sight Verification

Whenever Lazy Theta* attempts to connect two non-adjacent nodes, the planner must determine whether the straight segment intersects any occupied cell.

The proposed implementation performs this verification using Bresenham's line algorithm, which incrementally traverses all grid cells crossed by the connecting segment.

Given two vertices

$$\mathbf{v}_i = (x_i, y_i)$$

and

$$\mathbf{v}_j = (x_j, y_j),$$

the algorithm returns

$$LOS(\mathbf{v}_i, \mathbf{v}_j) = \begin{cases} 1, & \text{if all traversed cells are free} \\ 0, & \text{otherwise.} \end{cases}$$

Only visible connections are incorporated into the final path.

4.5 Path Optimality

The principal advantage of Lazy Theta* is its ability to generate paths that closely approximate the true Euclidean shortest path without requiring post-processing.

Compared to conventional A*, the resulting trajectories typically contain

- fewer waypoints,
- fewer heading changes,
- shorter travel distance,
- lower computational effort.

These characteristics are particularly important for multirotor UAVs, whose energy consumption increases significantly during aggressive manoeuvres.

4.6 Trajectory Smoothing

Although the path generated by Lazy Theta* is geometrically valid, it remains unsuitable for direct execution because abrupt heading changes produce discontinuous velocity profiles.

To overcome this limitation, the waypoint sequence is approximated using cubic B-Spline interpolation.

Given a sequence of control points

$$P_0, P_1, \ldots, P_n,$$

the resulting trajectory is expressed as

$$C(u) = \sum_{i=0}^{n} N_{i,3}(u) P_i,$$

where

$$N_{i,3}(u)$$

denotes the cubic B-Spline basis functions.

Unlike polynomial interpolation, B-Splines possess local support, meaning that modifying one control point affects only a limited portion of the curve.

This property considerably simplifies trajectory updates whenever the destination changes during flight.

4.7 Trajectory Properties

The generated trajectories exhibit several properties that are desirable for autonomous flight:

- continuous position,
- continuous velocity,
- continuous acceleration,
- reduced steering effort,
- smoother yaw transitions,
- improved flight stability.

These characteristics minimise oscillatory behaviour in the flight controller while simultaneously reducing mechanical stress on the propulsion system.

4.8 Planning Pipeline

Algorithm 4.8 summarises the complete planning procedure executed after TOPA selects the communication-aware destination.

[H] Motion Planning Pipeline
Current UAV position, target position, occupancy grid
Smooth collision-free trajectory
Inflate occupancy grid
Construct graph representation
Execute Lazy Theta* search
Extract waypoint sequence
Apply cubic B-Spline interpolation
Sample smooth trajectory
Transmit trajectory to the flight controller

4.9 Computational Complexity

Let V denote the number of graph vertices and E the number of graph edges.

The graph search performed by Lazy Theta* has complexity

$$\mathcal{O}(E \log V),$$

which is equivalent to classical A* when implemented using a priority queue.

Trajectory smoothing operates only on the final waypoint sequence containing k control points, resulting in linear complexity

$$\mathcal{O}(k).$$

Since

$$k \ll V,$$

the computational cost of spline generation is negligible compared with graph exploration.

Consequently, the complete planning pipeline remains suitable for real-time operation even in relatively large occupancy grids.

4.10 Discussion

The proposed planning strategy combines the strengths of graph-search algorithms with continuous trajectory generation.

Lazy Theta* efficiently computes short collision-free paths while naturally reducing unnecessary heading changes through any-angle navigation. The subsequent B-Spline interpolation transforms the discrete waypoint sequence into a dynamically feasible trajectory that can be directly executed by the flight controller.

This two-stage approach achieves an effective compromise between computational efficiency, path optimality and flight smoothness, making it particularly suitable for real-time UAV repositioning missions in cluttered environments.

5 System Implementation and Integration

The proposed autonomous repositioning framework has been implemented using the Robot Operating System 2 (ROS2), which provides a modular middleware for distributed robotic applications. ROS2 was selected due to its support for asynchronous communication, real-time execution, and seamless integration with modern autonomous systems.

Rather than implementing the framework as a monolithic application, each functional component was developed as an independent ROS2 node. This modular architecture improves scalability, simplifies debugging, and enables individual modules to be upgraded without affecting the remaining system.

Figure **??** illustrates the software architecture adopted throughout this work.

5.1 ROS2 Node Architecture

The complete system is composed of several specialised ROS2 nodes, each responsible for a well-defined task within the navigation pipeline.

The principal nodes are:

- User Tracking Node
- Environment Manager
- TOPA Optimisation Node
- Path Planner Node
- Trajectory Generator
- Mission Controller
- MAVLink Interface
- Telemetry Logger

Communication between nodes is performed through ROS2 topics, while service calls are employed whenever synchronous requests are required.

This design allows each module to execute independently while maintaining deterministic data exchange across the system.

5.2 User Tracking

The User Tracking Node continuously receives the estimated position of the mobile user.

Although the current implementation employs simulated position data generated within the Software-in-the-Loop (SITL) environment, the node has been designed independently of the localisation method.

Consequently, GPS receivers, Ultra-Wideband (UWB) systems, visual localisation algorithms or external tracking infrastructures can be incorporated without requiring modifications to the remainder of the framework.

Whenever a new user position is received, the node immediately publishes the updated coordinates to the positioning subsystem.

5.3 Environment Manager

The Environment Manager maintains the occupancy grid representing the operational scenario.

At system initialisation, the environment map is loaded into memory and converted into the internal data structures required by the planning algorithms.

The node provides several services to the remaining modules, including

- obstacle queries;
- occupancy verification;
- Line-of-Sight validation;
- map resolution conversion;
- coordinate transformations.

By centralising all map operations within a dedicated component, the remaining nodes remain independent of the underlying environment representation.

5.4 Communication Optimisation

The Communication Optimisation Node implements the Traffic- and Obstacle-aware Positioning Algorithm (TOPA).

Whenever the user's position changes, the node performs the following operations:

1. Generate candidate UAV positions.
2. Verify obstacle-free visibility.
3. Evaluate communication quality.
4. Compute the optimisation score.
5. Select the highest-scoring candidate.

The selected position is then published as the next navigation objective.

5.5 Global Planner

The Global Planner receives the current UAV position together with the destination generated by TOPA.

The planner first inflates the occupancy grid according to the UAV safety radius before executing Lazy Theta*.

The resulting waypoint sequence is immediately forwarded to the trajectory generation module.

Separating destination optimisation from path planning considerably simplifies the overall software architecture while allowing different planning algorithms to be integrated in future work.

5.6 Trajectory Generation

The Trajectory Generator converts the discrete waypoint sequence into a smooth continuous trajectory using cubic B-Spline interpolation.

The generated trajectory is uniformly sampled to produce a sequence of reference positions compatible with the flight controller.

In addition to improving flight smoothness, this stage significantly reduces abrupt velocity changes that would otherwise be introduced by piecewise linear paths.

5.7 Mission Controller

The Mission Controller coordinates the execution of the complete autonomous navigation pipeline.

Rather than continuously sending individual waypoints, the controller supervises mission execution through a finite-state machine (FSM).

The principal operational states are

- Idle;
- Waiting for User Position;
- TOPA Optimisation;
- Path Planning;
- Trajectory Generation;
- Flight Execution;
- Mission Completed.

Transitions between states occur automatically whenever the corresponding subsystem completes its execution.

This strategy prevents race conditions while ensuring a predictable execution sequence.

5.8 MAVLink Communication

Communication with the UAV autopilot is performed using the MAVLink protocol.

MAVLink provides a lightweight message-based communication framework widely adopted by open-source flight controllers such as ArduPilot and PX4.

The MAVLink Interface node is responsible for

- receiving telemetry data;
- monitoring vehicle state;
- uploading missions;

- sending waypoint commands;
- monitoring flight execution.

Vehicle telemetry, including position, attitude, velocity and battery status, is continuously published to the ROS2 ecosystem.

Conversely, high-level navigation commands generated by the planning subsystem are translated into MAVLink mission messages understood by the autopilot.

5.9 ArduPilot SITL Integration

To validate the proposed framework without requiring physical hardware, all experiments were conducted using the ArduPilot Software-in-the-Loop (SITL) simulator.

The simulator reproduces the behaviour of the complete flight controller while allowing rapid experimentation under controlled conditions.

This approach provides several advantages:

- safe testing;
- repeatable experiments;
- rapid debugging;
- deterministic execution;
- simplified parameter tuning.

Since the communication layer relies exclusively on MAVLink, migrating from simulation to a physical UAV requires minimal software modifications.

5.10 Data Logging

Every experiment is automatically recorded using a dedicated Telemetry Logger.

The logger stores

- UAV position;
- user position;

- planned trajectory;
- optimisation score;
- execution time;
- communication status;
- mission events.

The recorded data are later employed for quantitative performance evaluation and experimental comparison.

5.11 Execution Pipeline

The complete software pipeline executed during operation is summarised below:

1. Receive updated user position.
2. Execute TOPA optimisation.
3. Compute optimal UAV destination.
4. Execute Lazy Theta* planning.
5. Generate B-Spline trajectory.
6. Upload trajectory through MAVLink.
7. Execute autonomous flight.
8. Record telemetry.
9. Wait for the next user update.

This cycle is continuously repeated throughout the mission, enabling the UAV to adapt its position whenever communication conditions change.

5.12 Discussion

One of the principal strengths of the proposed implementation is its modularity.

Because every subsystem operates as an independent ROS2 node, individual algorithms may be replaced without affecting the remaining architecture. For example, the current TOPA optimisation module may be substituted by a learning-based communication planner, or the Lazy Theta* planner may be replaced by sampling-based methods such as RRT*.

Similarly, the MAVLink communication layer abstracts the underlying flight controller, allowing the framework to operate with either ArduPilot or PX4 while preserving the same high-level software architecture.

This modular implementation therefore provides a robust foundation for future extensions involving dynamic obstacle avoidance, multi-UAV coordination, online environment mapping and adaptive communication-aware navigation.

6 Experimental Setup

The proposed communication-aware autonomous navigation framework was experimentally validated in a controlled Software-in-the-Loop (SITL) environment. The objective of the experiments was to evaluate the ability of the system to continuously maintain Line-of-Sight (LoS) communication while autonomously generating collision-free trajectories in cluttered environments.

Rather than focusing exclusively on trajectory generation, the experiments were designed to assess the complete autonomous pipeline, including communication-aware positioning, global path planning, trajectory smoothing, and autonomous flight execution.

6.1 Simulation Environment

All experiments were conducted using the ArduPilot Software-in-the-Loop (SITL) simulator integrated with ROS2 through the MAVLink communication protocol.

The simulation environment reproduces the behaviour of a real multirotor UAV while allowing repeatable experiments under controlled operating conditions.

The complete software stack consists of

- ROS2 Humble Hawksbill;
- ArduPilot SITL;
- MAVLink communication protocol;
- MAVROS interface;
- Python implementation of the planning algorithms.

This configuration enables the autonomous navigation framework to operate exactly as it would on a physical UAV while eliminating hardware-related risks during development.

6.2 Environment Description

The experimental environment consists of a previously mapped indoor scenario represented as a two-dimensional occupancy grid.

Static obstacles reproduce walls and structural elements commonly encountered in industrial facilities and urban environments.

The occupancy grid is generated before mission execution and remains fixed throughout each experiment.

Obstacle inflation is applied using a safety margin corresponding to the physical dimensions of the UAV.

This conservative representation guarantees that all generated trajectories maintain an adequate clearance from surrounding obstacles.

6.3 Mission Scenario

Each experiment begins with the UAV positioned at an initial location while a mobile ground user occupies an arbitrary position inside the mapped environment.

The experimental procedure follows the sequence below:

1. The ground-user position is received.
2. TOPA computes the optimal communication position.
3. Lazy Theta* generates a collision-free path.
4. B-Spline interpolation produces a smooth trajectory.

5. The trajectory is transmitted to ArduPilot.
6. The UAV autonomously executes the manoeuvre.
7. The process repeats after the user changes position.

This procedure emulates a realistic communication-support mission in which the UAV continuously follows a moving user while preserving wireless visibility.

6.4 Evaluation Metrics

The proposed framework is evaluated according to several quantitative performance indicators.

6.4.1 Path Length

The total travelled distance is computed as

$$L = \sum_{i=1}^{N-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|,$$

where

$$\mathbf{p}_i$$

denotes consecutive trajectory samples.

Shorter trajectories generally correspond to lower flight time and reduced energy consumption.

6.4.2 Planning Time

Real-time operation requires rapid trajectory generation.

The planning time is measured as

$$T_p = T_{finish} - T_{start},$$

including

- TOPA optimisation;
- Lazy Theta* execution;
- B-Spline generation.

6.4.3 Trajectory Smoothness

Trajectory smoothness is evaluated by analysing the continuity of heading variations and curvature along the generated path.

Smooth trajectories reduce aggressive manoeuvres and improve controller stability during autonomous flight.

6.4.4 Communication Visibility

The primary objective of the framework is maintaining unobstructed communication.

For every trajectory sample, Line-of-Sight is verified between the UAV and the ground user.

The visibility ratio is computed as

$$R_{LOS} = \frac{N_{visible}}{N_{total}},$$

where

$$N_{visible}$$

represents the number of trajectory samples preserving Line-of-Sight.

6.4.5 Mission Completion

Mission success is defined by satisfying the following conditions:

- collision-free navigation;
- successful arrival at the destination;
- continuous autonomous execution;
- preserved communication visibility.

6.5 Experimental Parameters

Table 1 summarises the principal parameters adopted throughout the experimental evaluation.

6.6 Experimental Procedure

Multiple autonomous missions were executed using different user positions distributed throughout the environment.

Each experiment followed exactly the same execution pipeline to ensure repeatability.

For every mission, telemetry data, planning time, optimisation scores, generated trajectories and communication status were automatically recorded using the ROS2 logging infrastructure.

The resulting datasets were subsequently analysed offline to quantify the performance of the proposed framework.

6.7 Discussion

The adopted experimental methodology enables a comprehensive evaluation of the complete autonomous repositioning framework rather than isolated software components.

By integrating communication-aware positioning, autonomous path planning and flight execution within the same experimental pipeline, the evaluation closely reproduces the behaviour expected during real-world deployment.

Furthermore, the modular implementation allows future experiments to incorporate physical UAV platforms, dynamic obstacle avoidance, online mapping and multiple simultaneous users without requiring significant modifications to the experimental protocol.

7 Results and Discussion

This section presents the experimental evaluation of the proposed communication-aware autonomous navigation framework. The objective is to assess both the effectiveness of the individual system components and the overall behaviour of the integrated architecture during autonomous UAV repositioning missions.

The evaluation focuses on four main aspects:

- communication-aware destination selection;
- path planning efficiency;
- trajectory smoothness;

- overall autonomous mission execution.

7.1 Communication-Aware Positioning Performance

The first experiment evaluates the ability of the Traffic- and Obstacle-aware Positioning Algorithm (TOPA) to identify UAV locations that preserve Line-of-Sight communication while remaining accessible to the navigation subsystem.

For each user position, multiple candidate UAV locations were generated and evaluated according to the objective function described in Section III.

Figure **??** illustrates one representative optimisation scenario.

The optimisation process consistently selected positions that maintained unobstructed visibility while avoiding locations surrounded by obstacles.

Unlike nearest-neighbour positioning approaches, TOPA successfully prioritised communication quality over purely geometric proximity.

These results demonstrate that integrating environmental awareness into the positioning process significantly improves the quality of the selected UAV destinations.

7.2 Path Planning Evaluation

After selecting the communication-aware destination, the Lazy Theta* planner computed collision-free trajectories connecting the UAV's current position to the selected target.

Figure **??** presents an example of a planned trajectory.

The generated paths successfully avoided all obstacles while producing significantly fewer heading changes than conventional grid-based planners.

Because Lazy Theta* performs any-angle planning, the resulting trajectories closely approximated the Euclidean shortest path, reducing unnecessary detours.

No collisions were observed during any of the executed missions.

7.3 Trajectory Smoothing

Although Lazy Theta* generated geometrically efficient paths, the resulting waypoint sequence still contained abrupt direction changes unsuitable for direct UAV navigation.

Applying cubic B-Spline interpolation substantially improved trajectory continuity.

Figure **??** compares the original waypoint sequence with the final smooth trajectory.

The interpolation process removed sharp corners while preserving obstacle clearance and overall path geometry.

The resulting trajectories exhibited continuous curvature and smoother heading transitions, reducing the control effort required from the flight controller.

7.4 Autonomous Flight Execution

The complete navigation pipeline was evaluated through multiple autonomous repositioning missions executed inside the ArduPilot Software-in-the-Loop environment.

For every experiment, the UAV successfully completed the following sequence:

1. receive updated user position;
2. compute communication-aware destination;
3. generate collision-free path;
4. smooth the trajectory;
5. execute autonomous flight.

No software failures or mission interruptions were observed during testing.

The ROS2 middleware maintained reliable communication between all system components throughout the execution.

7.5 Planning Performance

The computational cost of the proposed framework remained compatible with real-time operation.

Table 2 summarises the principal performance indicators obtained during the experimental evaluation.

Although absolute execution times depend on the computational platform and environment size, all experiments completed within the requirements for online autonomous navigation.

Table 1: Experimental Parameters

Parameter	Value
Planner	Lazy Theta*
Trajectory Generator	Cubic B-Spline
Positioning Algorithm	TOPA
Environment	Occupancy Grid
Communication	MAVLink
Middleware	ROS2
Flight Controller	ArduPilot SITL
Vehicle Type	Quadrotor

Table 2: Representative Experimental Results

Metric	Result
Successful Missions	100%
Collision-Free Paths	100%
LoS Preservation	High
Trajectory Smoothness	Improved
Planning Time	Real-Time
ROS2 Communication	Stable
MAVLink Execution	Successful

7.6 Discussion

The experimental results validate the effectiveness of the proposed hierarchical navigation strategy.

Rather than directly planning trajectories towards the user's current location, the system first optimises the communication objective and subsequently computes a collision-free trajectory.

This separation considerably simplifies the navigation problem while naturally incorporating communication requirements into autonomous flight planning.

The combination of TOPA and Lazy Theta* proved particularly effective.

TOPA consistently selected destinations that maximised communication visibility, whereas Lazy Theta* generated efficient collision-free paths with relatively low computational cost.

The subsequent B-Spline interpolation further improved trajectory quality by eliminating discontinuities that would otherwise introduce aggressive flight manoeuvres.

From a software perspective, the ROS2-based architecture demonstrated excellent modularity.

Each subsystem operated independently while exchanging information through standard ROS2 communication interfaces.

Consequently, the proposed framework can be easily extended to incorporate additional functionalities such as dynamic obstacle avoidance, simultaneous localisation and mapping (SLAM), multi-UAV coordination or adaptive communication models.

7.7 Limitations

Despite the encouraging results, several limitations remain.

First, the current implementation assumes a static occupancy grid.

Dynamic obstacles are therefore not explicitly considered during path planning.

Second, communication quality is approximated through Line-of-Sight visibility rather than detailed radio propagation models.

Although this assumption is reasonable for many UAV communication scenarios, incorporating realistic channel models could further improve positioning accuracy.

Finally, all experiments were conducted within a Software-in-the-Loop simulation.

Although SITL closely reproduces the behaviour of physical UAV platforms, additional validation using real hardware will be necessary before deployment in operational environments.

Overall, the obtained results demonstrate that the proposed framework successfully integrates communication-aware positioning, autonomous navigation and trajectory generation into a unified real-time system capable of supporting reliable UAV communication missions in cluttered environments.